PyTorch 简介

杰克科技 杭州分公司

2026.04.30

目录

CONCENTS

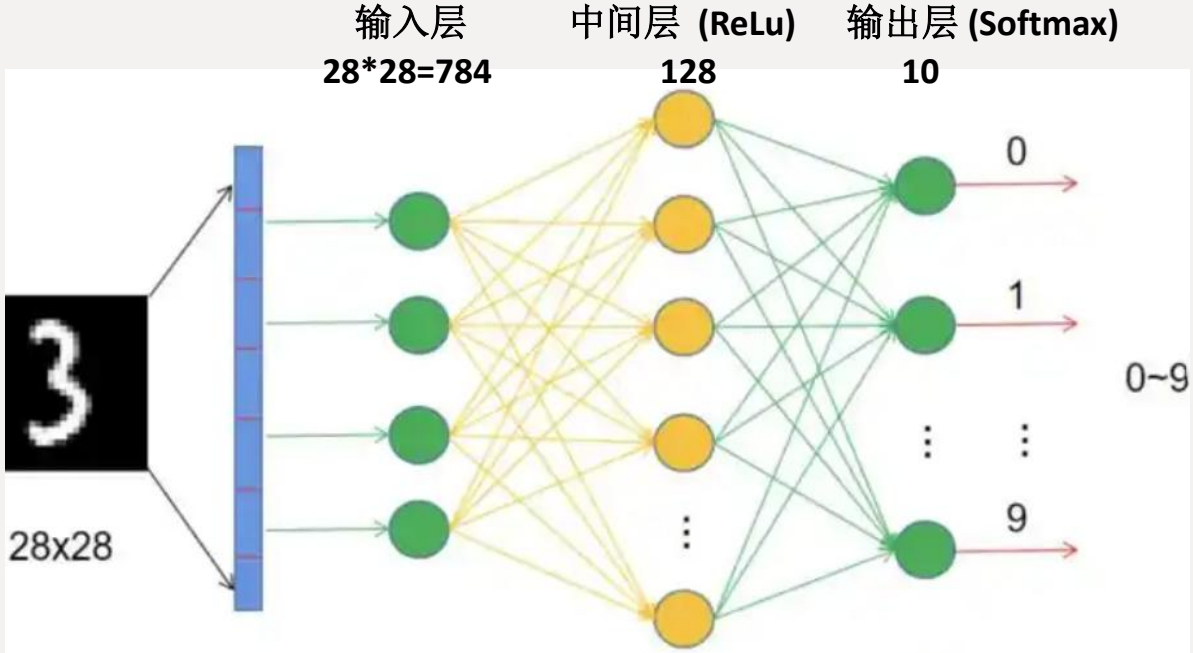
1 PyTorch 基础知识

2 案例分析：采用PyTorch构建神经网络

1 PyTorch 基础知识

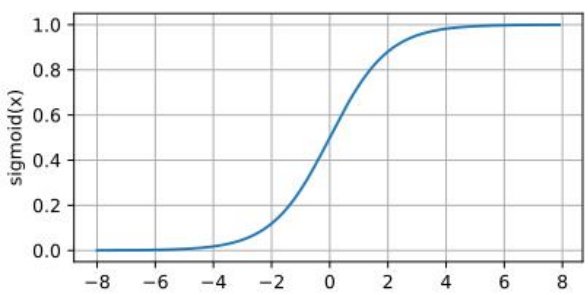
神经网络结构

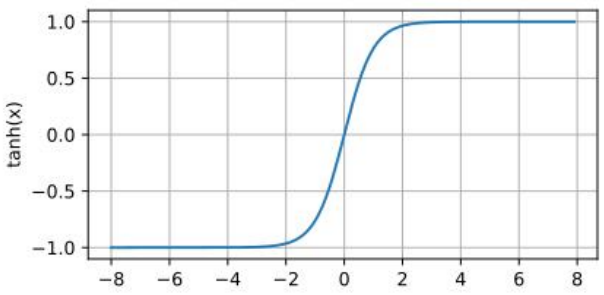
一个简单神经网络示例结构包含三层神经元，分别组成输入层、中间隐藏层和输出层。输入输出层神经元数量根据任务确定，中间层通常可自由指定。输入数据经中间一层或多层的隐藏层训练得到输出结果。

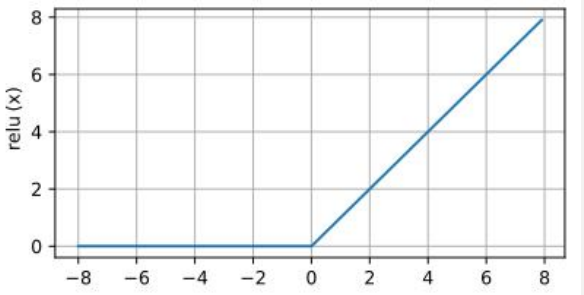


激活函数

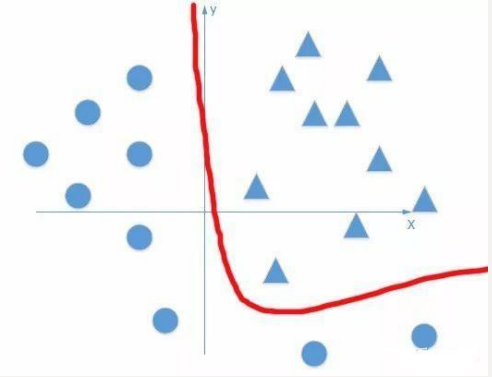
激活函数 $h_{k+2} = f(h_{k+1})$ 是在神经网络的神经元上运行的函数，该函数把神经元输入信号映射到输出端。常见的激活函数包括Sigmoid, tanh, ReLu等非线性函数，其输入-输出关系见下图。







在很多分类应用中，采用任意直线都无法解决如右图的圆圈和三角分类问题。只有采用非线性激活函数的神经网络才能拟合出右图红色分类曲线，顺利分类二种形状，故通常不采用线性函数作为神经元激活函数。



常见概念

- **优化器**：优化器又称优化算法，用来训练模型，计算和更新模型的网络参数使其逼近或者达到最优值的算法，比如常用的Adam优化器等。

- **损失函数**：计算神经网络每次迭代的前向计算结果与真实值的差距，常用的损失函数有均方差误差函数、均方根误差函数、交叉熵误差函数等，误差用于控制网络输出效果。

- **Epoch**：当一个神经网络已经遍历所有输入训练数据一次时，就完成一次epoch，即，它是全部样本数据（总数据量）输入网络训练的次数。例如，在采用10万组数据有监督训练一个神经网络时，经400次epoch训练其损失函数基本不再变化，达到一个稳定状态，则说明该神经网络已训练完毕。

- **Batch Size**：每轮预测和反向传播参数更新时使用的样本数。

- **Iteration**：完成一个epoch所需要的迭代次数，即需要处理多少个batch才能处理完一次全样本的数据。计算公式为：Iteration = 样本数据总数 / Batch Size。

备注：由于计算内存限制，通常batch size小于样本数据总数。不同batch可以在GPU并行计算；且可引入噪声，多次平均防止模型陷入局部最优，从而提升模型泛化能力。

损失函数 （选讲）

- 损失函数 l(y)衡量输出Y的优劣，表示为：

1. 均方误差(MSE) $$l(y) = ||y^* - y||^2$$

2. Log likelihood $$l(y) = \log \mathbb{P}[y^*|x]$$

- 损失由前向计算产生

$$x \xrightarrow[w_1]{} h_1 \xrightarrow[w_2]{} \dots \xrightarrow[w_n]{} h_n \xrightarrow[w_{n+1}]{} y \longrightarrow l(y)$$

- 损失的导数由反向计算产生

$$\begin{tikzcd}
\frac{\partial h_1}{\partial x} & \frac{\partial h_2}{\partial h_1} \arrow[l] \arrow[d] & \dots \arrow[l] & \frac{\partial y}{\partial h_n} \arrow[l] \arrow[d] & \frac{\partial l(y)}{\partial y} \arrow[l] \arrow[d] \\
 & \frac{\partial h_1}{\partial w_1} & \dots & \frac{\partial h_n}{\partial w_n} & \frac{\partial y}{\partial w_{n+1}}
\end{tikzcd}$$

梯度下降法 （选讲）

- 梯度下降法 也称为最快下降法，是一阶最优化算法 。 要使用梯度下降法找到一个函数的局部极小值，必须向函数上当前点对应梯度的反方向的规定步长距离点进行迭代搜索。

- 梯度下降方法基于以下的观察：如果实值函数 $F(x)$ 在点 a 处可微且有定义，那么函数 $F(x)$ 在点 a 沿着梯度相反的方向 $-\nabla F(a)$ 下降最快。对于另一个 b 点：

$$\mathbf{b} = \mathbf{a} - \gamma \nabla F(\mathbf{a})$$

对于 $\gamma > 0$ 为一个够小数值时成立，那么 $F(a) \geq F(b)$。

- 我们可以从函数 F 的初始估计值 $x0$ 出发，得到序列 $x0, x1, x2, x3 \cdots$, 使得

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \gamma_n \nabla F(\mathbf{x}_n),\ n \geq 0$$

如果顺利的话序列 $\{xn\}$ 收敛到期望的极小值。

随机梯度下降法 （选讲）

- 最小化期望损失

$$\mathcal{L}(w) = \mathbb{E}_x\left[l(y)\right]$$

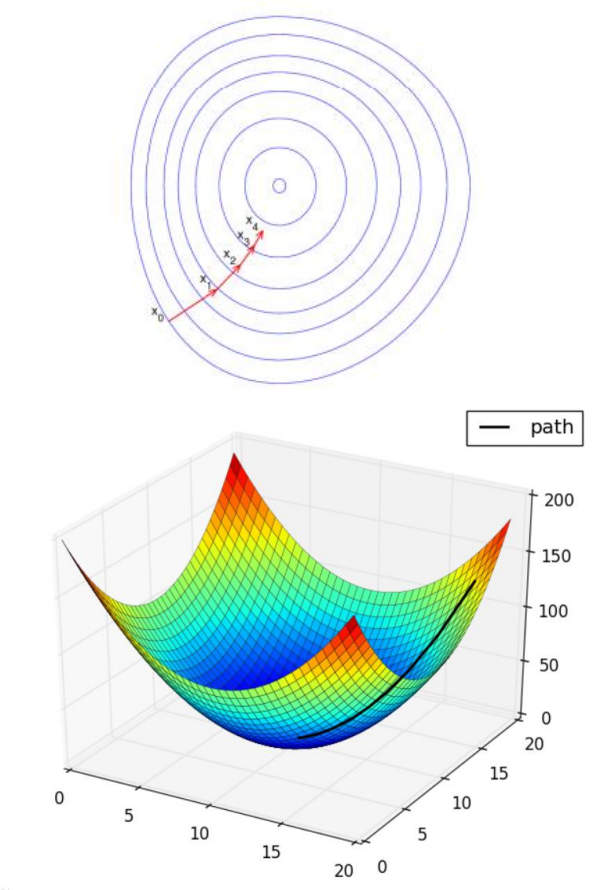
- 取得期望损失的梯度

$$\frac{\partial \mathcal{L}(w)}{\partial w} = \mathbb{E}_x\left[\frac{\partial l(y)}{\partial w}\right] = \mathbb{E}_x\begin{pmatrix}\frac{\partial l(y)}{\partial w^{(1)}}\\ \vdots \\ \frac{\partial l(y)}{\partial w^{(k)}}\end{pmatrix}$$

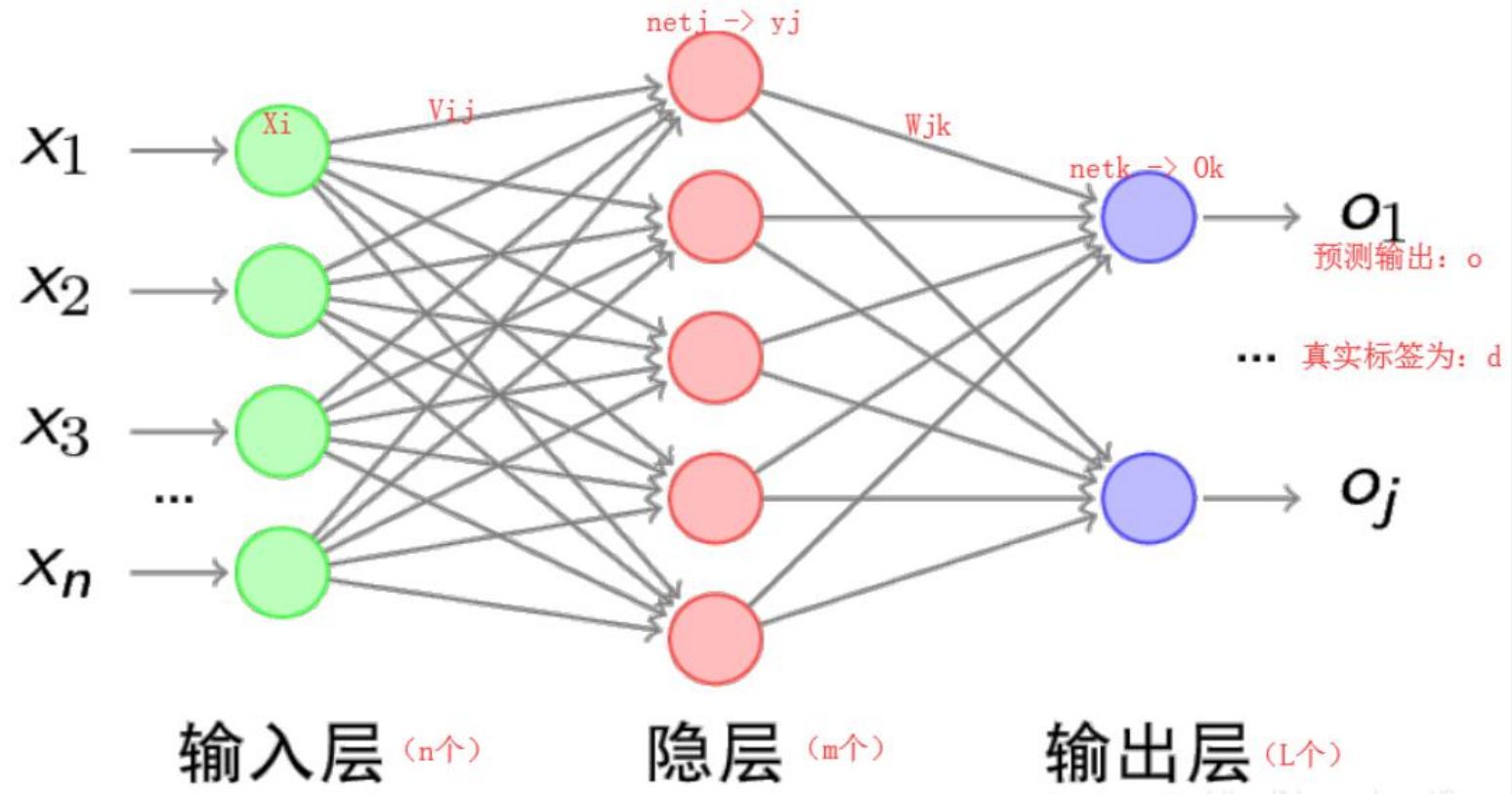
- 沿着反方向调整权值

$$\Delta w = -\frac{\alpha}{2}\alpha\frac{\partial l(y)}{\partial w}$$

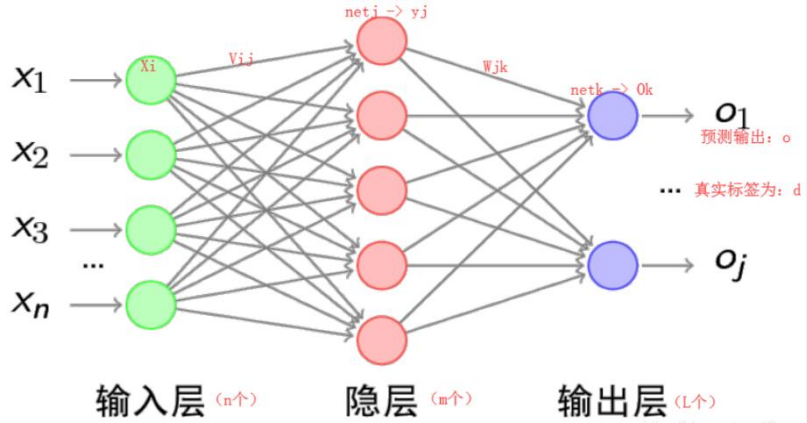
此处 α为单步参数



反向传播算法 （选讲）


netj -> yj
Vij
Wjk
netk -> Ok
Xi
x_1
x_2
x_3
...
x_n
o_1
预测输出：o
... 真实标签为：d
o_j
输入层（n个）
隐层（m个）
输出层（L个）

反向传播算法 （选讲）



输出层

$$E = \frac{1}{2}(d - O)^2 \qquad = \frac{1}{2}\sum_{\kappa=1}^{\ell}(d_k - o_k)^2$$

误差展开至隐层

$$E = \frac{1}{2}\sum_{\kappa=1}^{\ell}[d_\kappa - f(net_\kappa)]^2 \qquad = \frac{1}{2}\sum_{\kappa=1}^{\ell}[d_\kappa - f(\sum_{j=0}^{m}\omega_{j\kappa}y_j)]^2$$

展开至输入层

$$E = \frac{1}{2}\sum_{\kappa=1}^{\ell}[d_\kappa - f(\sum_{j=0}^{m}\omega_{j\kappa}f(net_j))]^2 \qquad = \frac{1}{2}\sum_{\kappa=1}^{\ell}[d_\kappa - f(\sum_{j=0}^{m}\omega_{j\kappa}f(\sum_{i=0}^{n}v_{ij}\chi_i))]^2$$

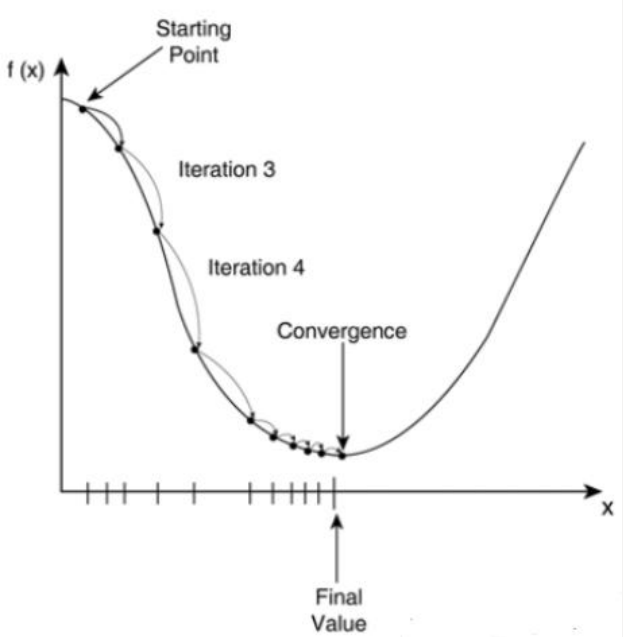
反向传播算法 （选讲）

1. 已知误差E，怎么调整权重让误差不断减小？
2. 显然误差E是权重w的函数，如何找到使得函数值最小的w？

Gradient Descent！

$$\Delta\omega_{j\kappa} = -\eta\frac{\partial E}{\partial\omega_{j\kappa}} j = 0, 1, 2, \ldots, m; \qquad \kappa = 1, 2, \ldots, \ell$$

$$\Delta v_{ij} = -\eta\frac{\partial E}{\partial v_{ij}} i = 0, 1, 2, \ldots, n; \qquad j = 1, 2, \ldots, m$$



有监督的深度学习 - Deep Supervised Learning （选讲）

- 深度神经网络　迄今取得很大成功，而PyTorch提供了采用深度神经网络拟合实际问题解法的算法框架。

- 简单的组成元素解决了监督学习难题

  - 采用深层网络作为函数拟合器
  - 定义了损失函数
  - 采用SGD端到端优化参数

- 可依据存储大小/数据量/计算量的不同要求进行很好的调整
- 解决了representation learning problem
- 针对图像、音频和语言等计算获得了state-of-the-art结果

2 案例分析：采用PyTorch构建神经网络

常见概念及网络超参数

- **优化器**：优化器又称优化算法，用来训练模型，计算和更新模型的网络参数使其逼近或者达到最优值的算法，比如常用的Adam，SGD优化器等。

- **损失函数**：计算神经网络每次迭代的前向计算结果与真实值的差距，常用的损失函数有均方差误差函数、均方根误差函数、交叉熵误差函数等，误差用于控制网络输出效果。

- **Epoch**：当一个神经网络已经遍历所有输入训练数据一次时，就完成一次epoch，即，它是全部样本数据（总数据量）输入网络训练的次数。例如，在采用10万组数据有监督训练一个神经网络时，经400次epoch训练其损失函数基本不再变化，达到一个稳定状态，则说明该神经网络已训练完毕。

- **Batch Size**：每轮预测和反向传播参数更新时使用的样本数。

- **Iteration**：完成一个epoch所需要的迭代次数，即需要处理多少个batch才能处理完一次全样本的数据。计算公式为：Iteration = 样本数据总数 / Batch Size。

- **Learning rate**：控制依据梯度而改变模型参数的尺度大小，比如选取0.001。

备注：由于计算内存限制，通常batch size小于样本数据总数。不同batch可以在GPU并行计算；且可引入噪声，多次平均防止模型陷入局部最优，从而提升模型泛化能力。

识别手写数字的案例

案例教程见课程网站：https://github.com/aitu-tech/EmboddiedIntelligenceLab/tree/main/src/module3

Aitu

谢　谢

Imagery Art Direction
摄影风格
新一代
制造力
People-centric
narratives